

Multi-Agent Reinforcement Learning & Agentic AI

Games: Models of Multi-Agent Interaction and Solution Concepts

Last week, we looked at single agents optimising MDPs via reinforcement learning.

This week, we will look at multiple agents acting in a shared environment with MARL, with the following topics covered over the next three lectures:

- Today: Multi-agent game models and solution concepts
- Wednesday & Thursday: Single-agent reductions, MARL challenges, MARL algorithms and applications.

Part 1: Game Models

- Normal-form games
- Stochastic games
- Partially observable stochastic games

Part 2: Assumptions

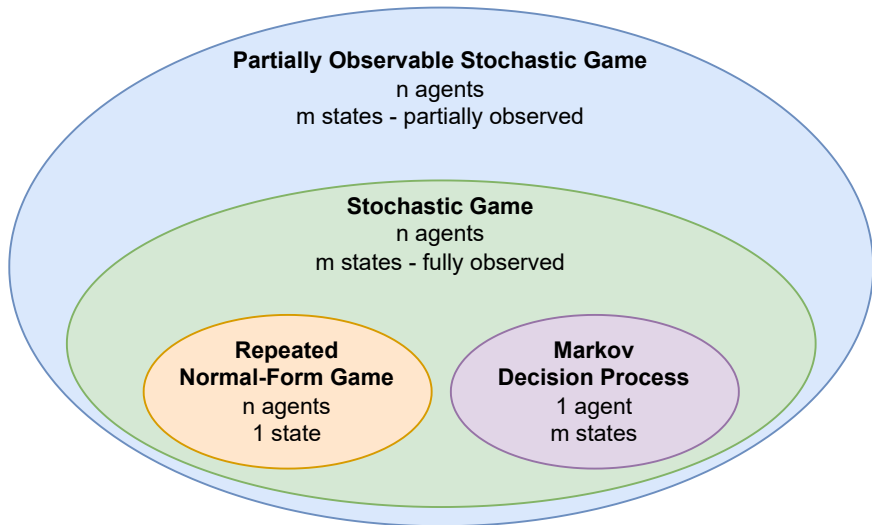
- Game theory vs MARL assumptions

Part 3: Solution Concepts

- Joint policy and expected return
- Equilibrium solution concepts

Game Models

Hierarchy of Games



Normal-Form Games

Normal-form games define a **single** interaction between two or more agents, providing a simple kernel for more general games to build upon.

Normal-Form Games

Normal-form games define a **single** interaction between two or more agents, providing a simple kernel for more general games to build upon.

Normal-form games are defined as a 3 tuple $(I, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I})$:

- I is a finite set of agents $I = \{1, \dots, n\}$

Normal-Form Games

Normal-form games define a **single** interaction between two or more agents, providing a simple kernel for more general games to build upon.

Normal-form games are defined as a 3 tuple $(I, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I})$:

- I is a finite set of agents $I = \{1, \dots, n\}$
- For each agent $i \in I$:
 - A_i is a finite set of actions
 - $\mathcal{R}_i$ is the reward function $\mathcal{R}_i : A \rightarrow \mathbb{R}$ where $A = A_1 \times \dots \times A_n$ (set of **joint** actions).

Normal-Form Game Process

In a normal-form game, there are **no time steps or states**. Agents choose an action and observe a reward.

The game proceeds as follows:

Normal-Form Game Process

In a normal-form game, there are **no time steps or states**. Agents choose an action and observe a reward.

The game proceeds as follows:

1. Each agent samples an action $a_i \in A_i$ with probability $\pi_i(a_i)$

Normal-Form Game Process

In a normal-form game, there are **no time steps or states**. Agents choose an action and observe a reward.

The game proceeds as follows:

1. Each agent samples an action $a_i \in A_i$ with probability $\pi_i(a_i)$
2. The resulting actions from all agents form a **joint action**, $a = (a_1, \dots, a_n)$

Normal-Form Game Process

In a normal-form game, there are **no time steps or states**. Agents choose an action and observe a reward.

The game proceeds as follows:

1. Each agent samples an action $a_i \in A_i$ with probability $\pi_i(a_i)$
2. The resulting actions from all agents form a **joint action**, $a = (a_1, \dots, a_n)$
3. Each agent receives a reward based on its **individual** reward function and the **joint action**, $r_i = \mathcal{R}_i(a)$

Games can be classified based on the relationship between the agents' reward functions.

- In **zero-sum games**, the sum of the agents' reward is always 0
i.e. $\sum_{i \in I} \mathcal{R}_i(a) = 0, \forall a \in A$

Games can be classified based on the relationship between the agents' reward functions.

- In **zero-sum games**, the sum of the agents' reward is always 0
i.e. $\sum_{i \in I} \mathcal{R}_i(a) = 0, \forall a \in A$
- In **common-reward** games, all agents receive the same reward ($R_i = R_j; \forall i, j \in I$)

Classes of Games

Games can be classified based on the relationship between the agents' reward functions.

- In **zero-sum games**, the sum of the agents' reward is always 0
i.e. $\sum_{i \in I} \mathcal{R}_i(a) = 0, \forall a \in A$
- In **common-reward** games, all agents receive the same reward ($R_i = R_j; \forall i, j \in I$)
- In **general-sum** games, there are no restrictions on the relationship between reward functions.

Matrix Games

Normal-form games with **2** agents are also called **matrix games** because they can be represented using reward matrices.

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Rock-Paper-Scissors

	A	B
A	10	0
B	0	10

Coordination Game

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

Prisoner's Dilemma

Matrix Games

Normal-form games with **2** agents are also called **matrix games** because they can be represented using reward matrices.

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Rock-Paper-Scissors
zero-sum

	A	B
A	10	0
B	0	10

Coordination Game

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

Prisoner's Dilemma

Matrix Games

Normal-form games with **2** agents are also called **matrix games** because they can be represented using reward matrices.

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Rock-Paper-Scissors
zero-sum

	A	B
A	10	0
B	0	10

Coordination Game
common-reward

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

Prisoner's Dilemma

Matrix Games

Normal-form games with **2** agents are also called **matrix games** because they can be represented using reward matrices.

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Rock-Paper-Scissors
zero-sum

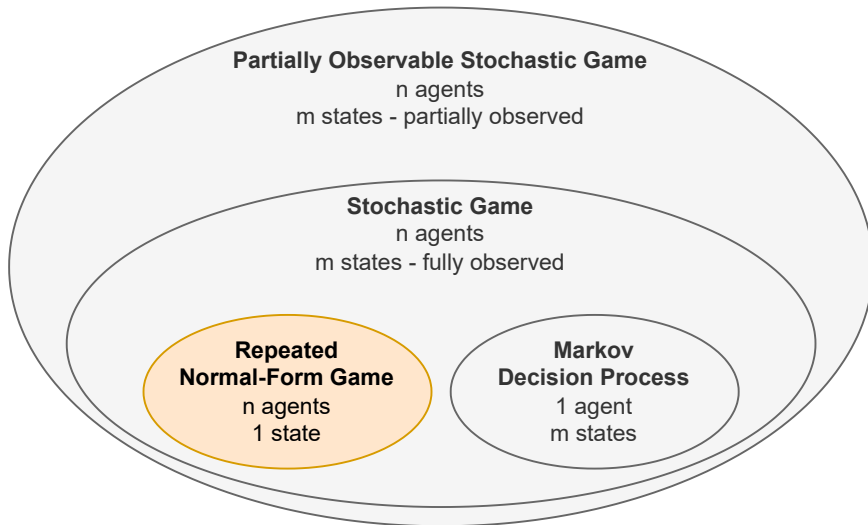
	A	B
A	10	0
B	0	10

Coordination Game
common-reward

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

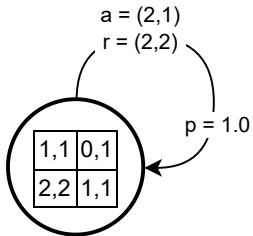
Prisoner's Dilemma
general-sum

Repeated Normal-Form Games



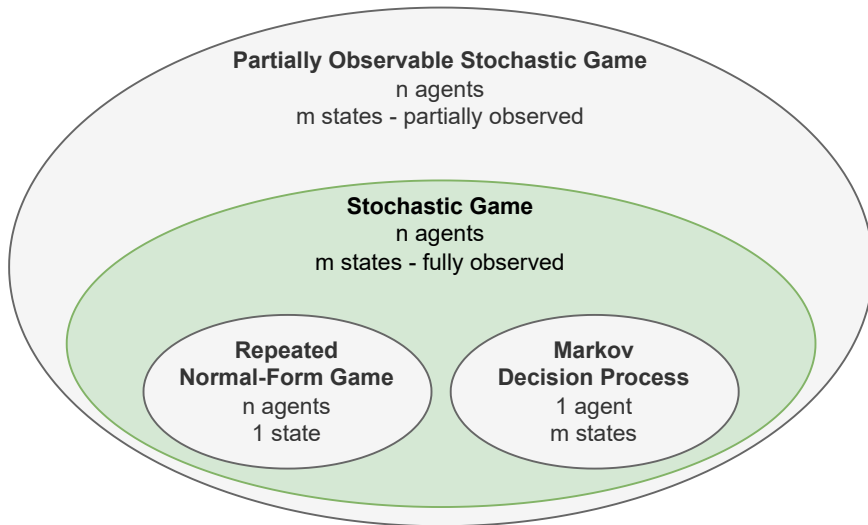
Repeated Normal-Form Games

To extend normal-form games to **sequential** multi-agent interaction, we can repeat the same game over T timesteps.



- At each time step t an agent i samples an action a_i^t
- The policy is now conditioned on a **joint-action** history $\pi_i(a_i^t|h^t)$ where $h^t = (a^0, \dots, a^{t-1})$
- In special cases, h^t contains n last joint actions. E.g. in a tit-for-tat strategy (Axelrod and Hamilton 1981), the policy is conditioned on a^{t-1}

Stochastic Games



Stochastic games introduce the notion of **states** and are defined as a 6 tuple $(I, S, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I}, \mathcal{T}, \mu)$

Stochastic Games

Stochastic games introduce the notion of **states** and are defined as a 6 tuple $(I, S, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I}, \mathcal{T}, \mu)$

- I is a finite set of agents

Stochastic Games

Stochastic games introduce the notion of **states** and are defined as a 6 tuple $(I, S, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I}, \mathcal{T}, \mu)$

- I is a finite set of agents
- S is a finite set of states with subset of terminal states $\bar{S} \subset S$

Stochastic Games

Stochastic games introduce the notion of **states** and are defined as a 6 tuple $(I, S, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I}, \mathcal{T}, \mu)$

- I is a finite set of agents
- S is a finite set of states with subset of terminal states $\bar{S} \subset S$
- For each agent $i \in I$:
 - A_i is a finite set of actions
 - $\mathcal{R}_i$ is the reward function $\mathcal{R}_i : S \times A \times S \rightarrow \mathbb{R}$ where A is the set of **joint** actions $A = A_1 \times \dots \times A_n$

Stochastic games introduce the notion of **states** and are defined as a 6 tuple $(I, S, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I}, \mathcal{T}, \mu)$

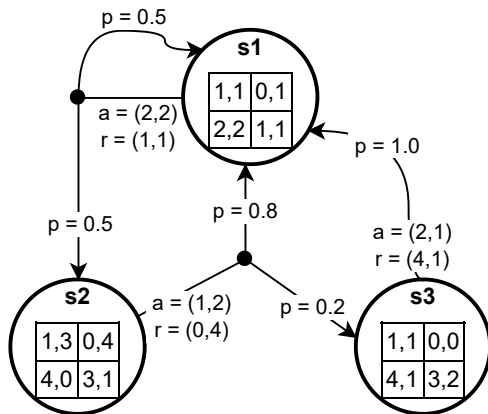
- I is a finite set of agents
- S is a finite set of states with subset of terminal states $\bar{S} \subset S$
- For each agent $i \in I$:
 - A_i is a finite set of actions
 - $\mathcal{R}_i$ is the reward function $\mathcal{R}_i : S \times A \times S \rightarrow \mathbb{R}$ where A is the set of **joint** actions $A = A_1 \times \dots \times A_n$
- $\mathcal{T}$ is the state transition function $\mathcal{T} : S \times A \times S \rightarrow [0, 1]$

Stochastic Games

Stochastic games introduce the notion of **states** and are defined as a 6 tuple $(I, S, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I}, \mathcal{T}, \mu)$

- I is a finite set of agents
- S is a finite set of states with subset of terminal states $\bar{S} \subset S$
- For each agent $i \in I$:
 - A_i is a finite set of actions
 - $\mathcal{R}_i$ is the reward function $\mathcal{R}_i : S \times A \times S \rightarrow \mathbb{R}$ where A is the set of **joint** actions
 $A = A_1 \times \dots \times A_n$
- $\mathcal{T}$ is the state transition function $\mathcal{T} : S \times A \times S \rightarrow [0, 1]$
- μ is the initial state distribution $\mu : S \rightarrow [0, 1]$

Stochastic Games - Continued



- Each **state** can be viewed as a **non-repeated normal-form game**
- Stochastic games can also be classified into: zero-sum, common-reward or general-sum
- The figure on the left shows a general-sum case

Stochastic Game Process

1. Initial state $s_0 \in S$ is samples from μ .

Stochastic Game Process

1. Initial state $s_0 \in S$ is samples from μ .
2. At time t each agent $i \in I$ observes the current state $s^t \in S$ and chooses an action $a_i^t \in A_i$ with probability $\pi_i(a_i^t | h^t)$

Stochastic Game Process

1. Initial state $s_0 \in S$ is samples from μ .
2. At time t each agent $i \in I$ observes the current state $s^t \in S$ and chooses an action $a_i^t \in A_i$ with probability $\pi_i(a_i^t|h^t)$
 - $h^t = (s^0, a^0, s^1, a^1, \dots, s^t)$ is the **state-action history** containing the current state s^t and past **joint** actions and states

Stochastic Game Process

1. Initial state $s_0 \in S$ is samples from μ .
2. At time t each agent $i \in I$ observes the current state $s^t \in S$ and chooses an action $a_i^t \in A_i$ with probability $\pi_i(a_i^t|h^t)$
 - $h^t = (s^0, a^0, s^1, a^1, \dots, s^t)$ is the **state-action history** containing the current state s^t and past **joint** actions and states
3. Given s^t and a^t , the game transitions into the next state $s^{t+1} \in S$ with probability $\mathcal{T}(s^{t+1}|s^t, a^t)$

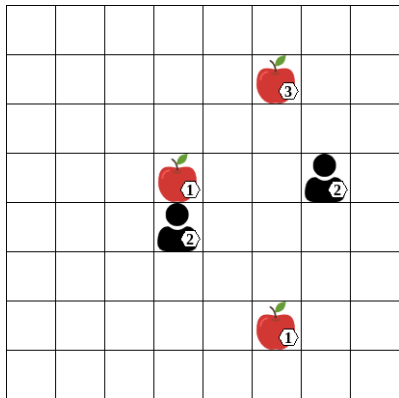
Stochastic Game Process

1. Initial state $s_0 \in S$ is samples from μ .
2. At time t each agent $i \in I$ observes the current state $s^t \in S$ and chooses an action $a_i^t \in A_i$ with probability $\pi_i(a_i^t|h^t)$
 - $h^t = (s^0, a^0, s^1, a^1, \dots, s^t)$ is the **state-action history** containing the current state s^t and past **joint** actions and states
3. Given s^t and a^t , the game transitions into the next state $s^{t+1} \in S$ with probability $\mathcal{T}(s^{t+1}|s^t, a^t)$
4. Each agent receives a reward according to their reward function $r_i^t = \mathcal{R}_i(s^t, a^t, s^{t+1})$

Stochastic Game Process

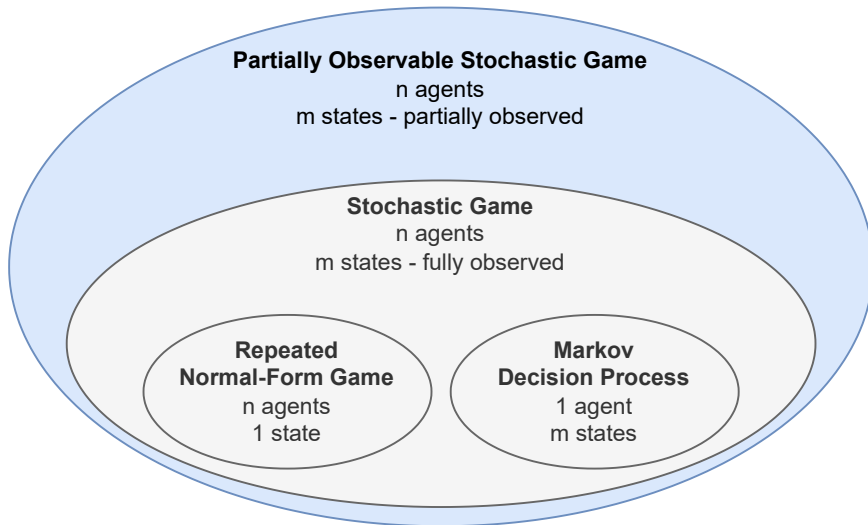
1. Initial state $s_0 \in S$ is samples from μ .
2. At time t each agent $i \in I$ observes the current state $s^t \in S$ and chooses an action $a_i^t \in A_i$ with probability $\pi_i(a_i^t|h^t)$
 - $h^t = (s^0, a^0, s^1, a^1, \dots, s^t)$ is the **state-action history** containing the current state s^t and past **joint** actions and states
3. Given s^t and a^t , the game transitions into the next state $s^{t+1} \in S$ with probability $\mathcal{T}(s^{t+1}|s^t, a^t)$
4. Each agent receives a reward according to their reward function $r_i^t = \mathcal{R}_i(s^t, a^t, s^{t+1})$
5. These steps are repeated until a terminal state $s^t \in \bar{S}$ is reached or a maximum number of T time steps is completed

Example: Level-Based Foraging



- $s \in S$: vector of x-y positions for agents/items, binary collection flags, levels for agents/items
- $a_i \in A_i$: move in four directions, collect item, or no operation (noop)
- $\mathcal{T}$: joint actions update state, e.g., two agents collecting an item switch its flag
- $\mathcal{R}$:
 - common-reward: +1 reward for any item collected by any agent
 - general-sum: +1 reward only for agents directly involved in item collection

Partially Observable Stochastic Games (POSG)



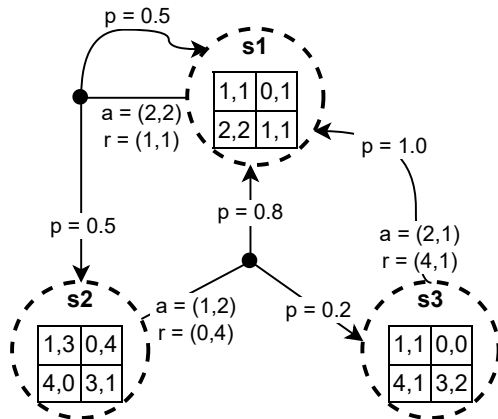
Partially Observable Stochastic Games (POSG)

At the top of the game model hierarchy, the most **general** model is the POSG

- POSGs represent complex decision processes with **incomplete information**
- Unlike in stochastic games, agents receive **observations** providing **incomplete information** about the state and agents' actions
- POSGs apply to scenarios where agents have limited sensing capabilities
 - ⇒ e.g. autonomous driving
 - ⇒ e.g. strategic games (e.g. card games) with private, hidden information

POSG Definition

POSG is defined in the same way stochastic games are, with two additions. Thus it is defined as a 8 tuple $(I, S, \{A_i\}_{i \in I}, \{\mathcal{R}_i\}_{i \in I}, \mathcal{T}, \mu, \{O_i\}_{i \in I}, \{\mathcal{O}_i\}_{i \in I})$



For each agent i we additionally define:

- a finite set of observation O_i
- an observation function $\{\mathcal{O}_i\}_{i \in I}$ such that $\mathcal{O}_i : A \times S \times O_i \rightarrow [0, 1]$ and $\forall a \in A, s \in S : \sum_{o_i \in O_i} \mathcal{O}_i(a, s, o_i) = 1$

1. Initial state s^0 sampled from $\mu(s)$

POSG Process

1. Initial state s^0 sampled from $\mu(s)$
2. At each t every agent i receives an **observation** $o_i^t \in O_i$, with probability given by the observation function $\mathcal{O}_i(o^t | s^t, a^{t-1})$

POSG Process

1. Initial state s^0 sampled from $\mu(s)$
2. At each t every agent i receives an **observation** $o_i^t \in O_i$, with probability given by the observation function $\mathcal{O}_i(o^t | s^t, a^{t-1})$
3. Each agent then chooses an action a_i^t based on its policy π_i , which is conditioned on the agent's observation history, $h_i = (o_i^0, \dots, o_i^t)$. All agents' actions give the joint action $a^t = (a_1^t, \dots, a_n^t)$

1. Initial state s^0 sampled from $\mu(s)$
2. At each t every agent i receives an **observation** $o_i^t \in O_i$, with probability given by the observation function $\mathcal{O}_i(o^t | s^t, a^{t-1})$
3. Each agent then chooses an action a_i^t based on its policy π_i , which is conditioned on the agent's observation history, $h_i = (o_i^0, \dots, o_i^t)$. All agents' actions give the joint action $a^t = (a_1^t, \dots, a_n^t)$
4. Given the joint action, the environment transitions into the next state s^{t+1} with probability $\mathcal{T}(s^{t+1} | s^t, a^t)$ and each agent receives a reward based on its reward function $r_i^t = \mathcal{R}_i(s^t, a^t, s^{t+1})$

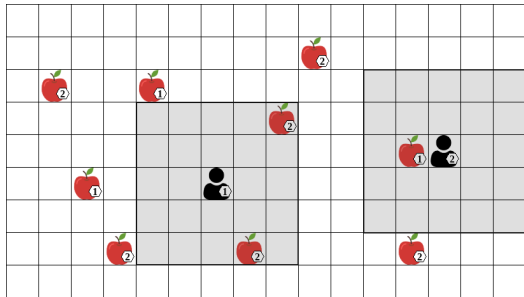
POSG Process

1. Initial state s^0 sampled from $\mu(s)$
2. At each t every agent i receives an **observation** $o_i^t \in O_i$, with probability given by the observation function $\mathcal{O}_i(o_i^t | s^t, a^{t-1})$
3. Each agent then chooses an action a_i^t based on its policy π_i , which is conditioned on the agent's observation history, $h_i = (o_i^0, \dots, o_i^t)$. All agents' actions give the joint action $a^t = (a_1^t, \dots, a_n^t)$
4. Given the joint action, the environment transitions into the next state s^{t+1} with probability $\mathcal{T}(s^{t+1} | s^t, a^t)$ and each agent receives a reward based on its reward function $r_i^t = \mathcal{R}_i(s^t, a^t, s^{t+1})$
5. This is done until a terminating state $s^t \in \bar{S}$ is reached or a maximum number of time steps is completed

The Observation Function

POSG can represent diverse observability conditions. For example:

- modeling noise by adding uncertainty in the possible observation
- to limit the visibility region of agents (see LBF example)

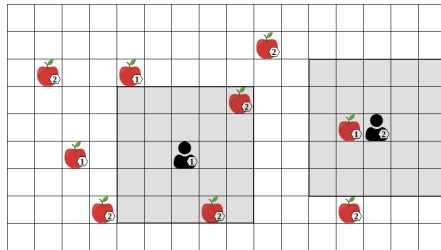


- Here, the agent can only see some parts of the state and joint action
- $o_i^t = (\bar{s}^t, \bar{a}^t)$ where $\bar{s}^t \subset s^t, \bar{a}^t \subset a^t$

Belief States

In partially observable settings, it becomes more challenging to infer optimal actions. For example:

- Optimal action for agent 1 is to move left towards level 1 apple
- But level 1 apple is not directly observable
- Agent 1 can hold a **belief state** b_i^t , a probability distribution over possible state $s \in S$
- Agent 1 might have seen the level-1 apple previously and can thus 'remember' its location



Single Agent Belief Update

To simplify, let's consider the single-agent perspective:

- The initial belief state is given by $b_i^0 = \mu$
- After taking action a_i^t and observing o_i^{t+1} , the belief state b_i^t is updated to b_i^{t+1} using a Bayesian update:

$$b_i^{t+1}(s') \propto \sum_{s \in S} b_i^t(s) \mathcal{T}(s'|s, a_i^t) \mathcal{O}_i(o_i^{t+1}|a_i^t, s')$$

Single Agent Belief Update

To simplify, let's consider the single-agent perspective:

- The initial belief state is given by $b_i^0 = \mu$
- After taking action a_i^t and observing o_i^{t+1} , the belief state b_i^t is updated to b_i^{t+1} using a Bayesian update:

$$b_i^{t+1}(s') \propto \sum_{s \in S} b_i^t(s) \mathcal{T}(s'|s, a_i^t) \mathcal{O}_i(o_i^{t+1}|a_i^t, s')$$

In MARL this type of update is typically **infeasible**:

- High-dimensional state spaces make storage and updates of beliefs intractable
- In MARL for POSG, agents assumed not to know $(S, \mathcal{T}, \mathcal{O}_i)$
- Deep learning can be used to approximate state information.

Assumptions in Games

Game Theory Assumption

- In game theory, we typically assume that all agents know all components of the game (**complete knowledge games**)
- Agents know all agents' **action spaces and reward functions**
- Knowledge of other agents' reward functions may be used for informing the agent's **best response** action (we will cover this in more depth in the next lecture)
- Knowledge of the transition function (T) allows for predicting state changes and **planning** actions multiple steps ahead

MARL Assumptions

- In MARL, we assume **limited knowledge**, i.e. no knowledge of transition function $\mathcal{T}$ and no knowledge of agents' reward functions $\mathcal{R}_i$
- Additional assumption can be added and specific knowledge of the game can be held **mutually** or **asymmetrically**
- We usually assume the **number of agents to be fixed**, although recent research has looked at *open* multi-agent systems, this will not be covered in these lectures

Dictionary: Reinforcement Learning $\leftrightarrow$ Game Theory

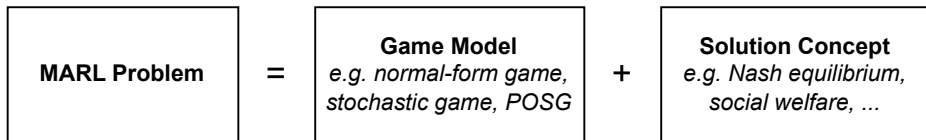
RL		GT
environment	$\leftrightarrow$	game
agent	$\leftrightarrow$	player
reward	$\leftrightarrow$	payoff, utility
policy	$\leftrightarrow$	strategy
deterministic X	$\leftrightarrow$	pure X
probabilistic X	$\leftrightarrow$	mixed X
joint X	$\leftrightarrow$	X profile

- **Environment/Game:** Model with actions, observations, rewards, state dynamics.
- **Agent/Player:** Decision-maker, possibly with specific roles.

- **Reward/Payoff, Utility:** Scalar value received after an action
- **Policy/Strategy:** Assigns probabilities to actions; 'pure strategy' may refer to actions
- **Deterministic X/Pure X:** Assigns probability 1 to X e.g. $X = \text{equilibrium}$ or policy
- **Probabilistic X/Mixed X:** Assigns probabilities ≤ 1 to X
- **Joint X/X Profile:** Tuple representing collective aspects, e.g., rewards or policies

Solution Concepts for Games

Solution Concepts for Games



What does it mean to act **optimally** in a multi-agent system?

- Maximizing returns of one agent is no longer enough to determine a solution
- We must consider the **joint policy** of all agents
- This is less straightforward, and there are many different solution concepts

Joint Policy and Expected Return

Joint Policy

The **joint policy** is the combination of all agents' policies:

$$\pi = (\pi_1, \dots, \pi_n)$$

Probability of a joint action under joint policy π is defined as:

$$\pi(a^\tau | h^\tau) = \prod_{j \in I} \pi_j(a_j^\tau | h_j^\tau)$$

Note

This definition assumes probabilistic independence between agents' policies.

Additional Notation

We will use additional notation:

- $\hat{h}^t = \{(s^\tau, o^\tau, a^\tau)_{\tau=0}^{t-1}, s^t, o^t\}$ is the **full history** containing:
 - s^τ , states up to $t - 1$
 - o^τ , joint observations up to $t - 1$
 - a^τ , joint actions up to $t - 1$
 - s^t and o^t at current time step t
- $\sigma(\hat{h}^t) = (o^0, \dots, o^t)$ returns the joint observation history from full history
- $\mathcal{O}(o^t | a^{t-1}, s^t)$ is the joint observation probability

History-Based Expected Return

Given a joint policy π , we can define the expected return for agent i under π as the probability-weighted sum of returns for agent i under all possible **full histories**.

- Let $\hat{H}$ be a set containing all full histories $\hat{h}^t$ for $t \rightarrow \infty$
- Expected return for agent i under joint policy π is given by:

$$U_i(\pi) = \sum_{\hat{h}^t \in \hat{H}} \Pr(\hat{h}^t | \pi) u_i(\hat{h}^t)$$

History-Based Expected Return – Continued

The probability of a full history $\Pr(\hat{h}^t|\pi)$ is:

$$\Pr(\hat{h}^t|\pi) = \mu(s^0)\mathcal{O}(o^0|\emptyset, s^0) \prod_{\tau=0}^{t-1} \pi(a^\tau|\hat{h}^\tau)\mathcal{T}(s^{\tau+1}|s^\tau, a^\tau)\mathcal{O}(o^{\tau+1}|a^\tau, s^{\tau+1})$$

$u_i(\hat{h}^t)$ is the discounted return for agent i in the **full history**

$$u_i(\hat{h}^t) = \sum_{\tau=0}^{t-1} \gamma^\tau R_i(s^\tau, a^\tau, s^{\tau+1})$$

Recursive Expected Return

Expected returns under a joint policy can also be defined *recursively*, analogously to Bellman recursion:

$$V_i^\pi(\hat{h}) = \sum_{a \in A} \pi(a \mid \sigma(\hat{h})) Q_i^\pi(\hat{h}, a)$$

Use this to define a Q -function for agent i as follows:

$$Q_i^\pi(\hat{h}, a) = \sum_{s' \in S} \mathcal{T}(s' \mid s(\hat{h}), a) \left[\mathcal{R}_i(s(\hat{h}), a, s') + \gamma \sum_{o' \in O} O(o' \mid a, s') V_i^\pi(\langle \hat{h}, a, s', o' \rangle) \right]$$

- $s(\hat{h})$ denotes the last state in $\hat{h}$ such that $s(\hat{h}) = s^t$

Recursive Expected Return – Continued

- $V_i^\pi(\hat{h})$ is the **value (expected return)** for agent i when agents follow **joint policy** π
- $Q_i^\pi(\hat{h}, a)$ is the expected return for agent i when agents execute **joint action** a after $\hat{h}$ and follow π thereafter
- Given the definition for $V_i^\pi(\hat{h})$ and $Q_i^\pi(\hat{h}, a)$, we can define the expected return for agent i at the initial state s^0 as:

$$U_i(\pi) = \mathbb{E}_{s_0 \sim \mu, o_0 \sim \mathcal{O}(\cdot | \emptyset, s_0)} [V_i^\pi(\langle s_0, o_0 \rangle)]$$

Equilibrium Solution Concepts

Best-response policy π_i maximizes expected return for agent i against a given set of policies for all other agents, $\pi_{-i} = (\pi_1, \dots, \pi_{i-1}, \pi_{i+1}, \dots, \pi_n)$

$$\text{BR}_i(\pi_{-i}) = \arg \max_{\pi_i} U_i(\langle \pi_i, \pi_{-i} \rangle)$$

where $\langle \pi_i, \pi_{-i} \rangle$ is the complete **joint policy**

Minimax is a solution concept defined for **two-agent zero-sum** games. Recall that one agent's reward is the negative of the other agent's reward.

- The existence of minimax solution in **normal-form games** was first proven by John von Neumann (1928)
- Minimax solutions also exist in **two-agent zero-sum stochastic games** with finite episode lengths, like chess and Go.

Minimax Definition

In a two-agent zero-sum game, a joint policy $\pi = (\pi_i, \pi_j)$ is a minimax solution if

$$U_i(\pi) = \max_{\pi'_i} \min_{\pi'_j} U_i(\pi'_i, \pi'_j) \quad (1)$$

$$= \min_{\pi'_j} \max_{\pi'_i} U_i(\pi'_i, \pi'_j) \quad (2)$$

$$= -U_j(\pi) \quad (3)$$

- Eq.1 is the minimum expected return agent i can guarantee against any opponent
- Eq.2 is the minimum expected return agent j can **force** on agent i
- A minimax solution is the **best response** to the **worst-case** opponent
- (π_i, π_j) is a minimax solution if $\pi_i \in \text{BR}_i(\pi_j)$ and $\pi_j \in \text{BR}_j(\pi_i)$

Minimax via Linear Programming

We can obtain a minimax solution for non-repeated zero-sum normal-form games by solving two linear programs, one for each agent.

$$\begin{array}{ll} \text{minimize} & U_j^* \\ \text{subject to} & \sum_{a_i \in A_i} \mathcal{R}_j(a_i, a_j) x_{a_i} \leq U_j^* \quad \forall a_j \in A_j \\ & x_{a_i} \geq 0 \quad \forall a_i \in A_i \\ & \sum_{a_i \in A_i} x_{a_i} = 1 \end{array}$$

- Minimizing agent j 's return U_j^*
- Such that no single action of agent j can receive a greater return than U_j^* when agent i follows $\pi_i(a_i) = x_{a_i}$

Nash Equilibrium

Nash equilibrium solution concept applies the idea of a **mutual best response** to general-sum games with two or more agents.

- No agent i can improve its expected returns by changing its policy π_i assuming other agents policies remain fixed:

$$\forall i, \pi'_i : U_i(\pi'_i, \pi_{-i}) \leq U_i(\pi)$$

- Each agent's policy is a **best response** to all other agent's policies
- John Nash (1950) proved the existence of such a solution for **general-sum non-repeated normal-form games**

Nash Equilibrium in Matrix Games

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

Prisoners Dilemma

	A	B
A	10	0
B	0	10

Coordination Game

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Rock Paper Scissors

Can you identify the Nash equilibria?

Nash Equilibrium in Matrix Games

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

Prisoners Dilemma
NE at D, D (-3, -3)

	A	B
A	10	0
B	0	10

Coordination Game

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Rock Paper Scissors

Nash Equilibrium in Matrix Games

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

Prisoners Dilemma
NE at D, D (-3, -3)

	A	B
A	10	0
B	0	10

Coordination Game
Two NE's at A, A (10) and
B, B (10)

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Rock Paper Scissors

Nash Equilibrium in Matrix Games

	C	D
C	-1,-1	-5,0
D	0,-5	-3,-3

Prisoners Dilemma
NE at D, D (-3, -3)

	A	B
A	10	0
B	0	10

Coordination Game
Two NE's at A, A (10) and
B, B (10)

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

Rock Paper Scissors
NE is to choose actions
uniformly at random with
expected return 0

Shortcomings of Equilibrium Solutions

Equilibrium solutions are standard solution concepts in MARL, but have **limitations**.

Shortcomings of Equilibrium Solutions

Equilibrium solutions are standard solution concepts in MARL, but have **limitations**.

- **Sub-optimality:**

- Nash equilibria do not always maximize expected returns
- E.g. in Prisoner's Dilemma, (D,D) is Nash but (C,C) yields higher returns

Shortcomings of Equilibrium Solutions

Equilibrium solutions are standard solution concepts in MARL, but have **limitations**.

- **Sub-optimality:**

- Nash equilibria do not always maximize expected returns
- E.g. in Prisoner's Dilemma, (D,D) is Nash but (C,C) yields higher returns

- **Non-uniqueness:**

- There can be multiple (even infinitely many) equilibria, each with different expected returns

Shortcomings of Equilibrium Solutions

Equilibrium solutions are standard solution concepts in MARL, but have **limitations**.

- **Sub-optimality:**

- Nash equilibria do not always maximize expected returns
- E.g. in Prisoner's Dilemma, (D,D) is Nash but (C,C) yields higher returns

- **Non-uniqueness:**

- There can be multiple (even infinitely many) equilibria, each with different expected returns

- **Incompleteness:**

- Equilibria for sequential games don't specify actions for **off-equilibrium paths**, i.e. paths not specified by equilibrium policy $\Pr(\hat{h}|\pi) = 0$
- If there is a temporary disturbance that leads to an off-equilibrium path, the equilibrium policy π does not specify actions to return to a **on-equilibrium** path

Refinement Concepts

Pareto Optimality

To address some of these limitations, we can add additional solution requirements such as **Pareto optimality**.

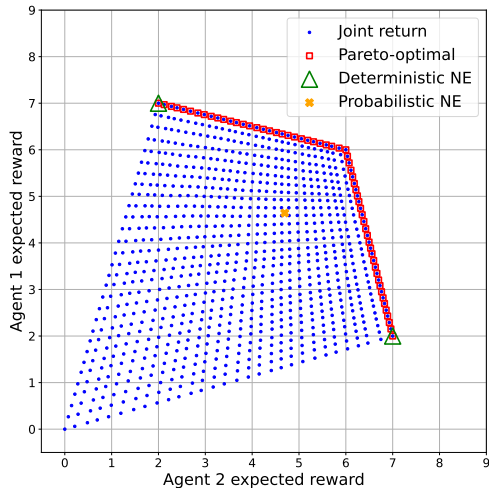
A joint policy π is **Pareto-optimal** if it is not **Pareto-dominated** by any other joint policy. A joint policy π is Pareto-dominated by another policy π' if

$$\forall i : U_i(\pi') \geq U_i(\pi) \quad \text{and} \quad \exists i : U_i(\pi') > U_i(\pi).$$

Intuition

A joint policy is **Pareto-optimal** if there is no other joint policy that improves the expected return for at least one agent without reducing the expected return for any other agent.

Pareto-Optimal Solution in the Chicken Game



	S	L
S	0,0	7,2
L	2,7	6,6

- Figure shows the discretized space of joint policies for Chicken matrix game
- Each blue dot represents the expected joint return obtained by a joint policy

Social Welfare and Fairness

To further constrain the space of desirable solutions, we can consider social welfare and fairness concepts.

Welfare optimality:

$$W(\pi) = \sum_{i \in I} U_i(\pi)$$

- A joint policy π is **welfare-optimal** if $\pi \in \arg \max_{\pi'} W(\pi')$

Fairness optimality:

$$F(\pi) = \prod_{i \in I} U_i(\pi)$$

- A joint policy π is **fairness-optimal** if $\pi \in \arg \max_{\pi'} F(\pi')$

Summary

We covered:

- Game Models
- Solution concepts for games
- Joint policy and expected return
- Equilibrium solution concepts
- Additional solution criteria

Next we'll cover:

- Single-agent reduction
- MARL challenges
- MARL algorithms and applications